

物件導向: 大寫。可重複大量使用。安全、穩定、快速(開發快速)。

物件 = 實例 = 範例。建立類別 = 制定型態(系統安全性)，使用類別 = 宣告。

例如: 型態人，有身高、體重等屬性。

age = list()，age是物件，list()是類別

方法: 類別中的函式，即同類別中每個物件都有的行為

屬性: 類別中的變數，即同類別中每個物件都有的性質

類別屬性: 所有類別物件共用

實例屬性: 當前物件

class Person: # self.是實例(物件)屬性，物件不同，實例屬性不同

count = 0 # 類別屬性，每個物件共享

def __init__(self, age = 0):

Person.count += 1 # 改變了類別屬性

self.name = "Jack" # 公開屬性

self.__age = age # 私有屬性，需從下方set_age方法做修改(連招)

def set_age(self, age):

if age < 0:

print("年齡不能是負數！")

else:

self.__age = age # 設置新值

wang = Person() # wang變成一個新物件，享有Person類別

self是wang或是Person()。wang.name。Person().name物件屬性要有括號

ming = wang # 與wang兩物件共享count類別屬性

只是指向wang並不是創建新物件，並不會讓count變成2

print(Person.count) # 印出1，只是提取Person類別裡的屬性，也可wang.count

print(Person().count) # 印出2，是因為Person()

print(Person().name) # 不能Person.name，因為name是實例(物件)屬性非類別屬性

del(刪除) wang # 因仍有ming存在指向Person()，故不會回收物件

當所有Person()的參照都沒有，Python才會回收該物件

import Person : import的時候會載入該模組(.py)一次。

Person.Person()，前一個是模組，後一個是類別；

如果是小寫person()則是方法，這是慣例命名

if __name__ == "__main__": 如果被import到別的程式，該條件不成立；

如果在該模組，__name__會是__main__；

如果因被別的程式載入而印出，則會印出Person

可直接新增實例屬性: 也可直接更改數值，但不建議

wang.newattr = 120

print(wang.newattr) # 120，不可用Person.newattr

一般函式是最簡單的一個物件，故而可被呼叫使用

封裝: 屬性不設在初始方法下，另設該類別其他方法，除本人外無法知道哪種方法

私有: 雙底線，不可從物件外直接調用，僅供物件內使用(上方)。def __name(self):(可用於方法)

保護: 單底線，保護，可繼承，雙底線不可繼承。可直接提取如Person._sex，也可被改變

保護是不嚴謹的隱藏。私有較保護嚴謹，無法被改變，倘若Person().__age = 10只會是新增一個__age的變數為10，不會改變age。age為物件屬性，非類別屬性的就是物件屬性。

需在內部設一方法(上方)來呼叫(正當)，或是print(a._Student__name)，Person()._Person__age

class Test:

def __init__(self, r=0, j=1) # Test(5,6)，參數被引入到初始化函式(建構式

方法過載: 方法的參數給定預設值

Python可以將函數作為參數，Java不行

某些前後有雙底線的方法 (即「魔法方法」或「特殊方法」) 會在特定情況下被自動執行

def __str__(self): # 這種形式的建構函式會自動執行，且大部分都需return

print(self.weight)

return "" # 必須要有return

類別方法: 前面教類別屬性，現在是類別方法。@classmethod，@是裝飾(修飾)器的標誌。

用類別方法僅是讓程式更好理解，將使用類別屬性的一般方法前面加個裝飾註記(不用類別方法也行)

@classmethod # 用cls而非self

def get_count(cls): # cls 代表類別本身

return cls.count # 返回Person.count(類別屬性)

class Fraction :

def __init__(self , n = 0 , d = 1):

self.num , self.den = n , d

def __str__(self):

if self.den == 1:

return str(self.num)

else:

return str(self.num) + '/' + str(self.den)

@classmethod

def fromstr(cls, fstr):

if fstr.isdigit():

num,den = int(fstr) , 1

else:

num,den = map(int , fstr.split('/'))

return cls(num,den) # 創建並回傳新的 Fraction 物件，等價

Fraction(num,den)

物件方法: 普通的方法，理論上可以用物件方法取代類別方法

靜態方法: @staticmethod，類別或物件共享的方法，沒用到物件跟類別屬性

class Math: # 第一個參數非物件(self)或類別(cls)

@staticmethod

def add(a, b):

return a + b

print(Math.add(3, 5))

Heroku: app.py -> host = '0.0.0.0' # 任何ip位址都能瀏覽。

規定部署(資料夾中的設定檔): Procfile、requirements(套件)、runtime(python版本3.11.11)

Heroku mariadb: resources -> data store -> 找DB ->

連結flask-jack -> 點資料庫連結 -> 複製名稱、密碼、database -> 更新python code

Heroku上面找需要做的前置作業

Heroku動態: 預設是linux所以要讓heroku安裝chrome driver

1.Heroku -> Settings -> Buildpacks

(1) heroku/python

(2) <https://github.com/heroku/heroku-buildpack-chrome-for-testing>

2.Heroku -> Settings -> Config Vars(環境變數)

>>> heroku run bash

key: GOOGLE_CHROME_BIN

/app/.chrome-for-testing/chrome-linux64/chrome

key: CHROMEDRIVER_PATH

/app/.chrome-for-testing/chromedriver-linux64/chromedriver